

Course syllabus

Undergraduate elective - 12 weeks - test fixture for ClassScribe

Department of Information Technology. This syllabus is a ClassScribe test fixture: every unit is written as lecture narration, not a keyword list, so syllabus ingestion has real prose to rewrite. Teaching hours assume 12 weeks, 3 hours of lecture plus 2 hours of lab.

Course details

- Code: ITC604 Credits: 4 Level: undergraduate, fifth semester
- Prerequisites: linear algebra (vectors, matrices), basic probability, Python
- Assessment: 20% labs, 30% midterm, 50% end-semester examination
- Text: Goodfellow, Bengio, Courville - Deep Learning (selected chapters)

Unit I: The perceptron and linear classification (8 hours)

*In this unit we treat the perceptron as a linear binary classifier. Inputs x_1 through x_n are multiplied by weights w_1 through w_n , a bias b is added, and a step activation decides the class. The decision boundary is the hyperplane $w \cdot x + b = 0$. We prove that a single perceptron can realise AND, OR and NOT, and we show Rosenblatt's perceptron criterion and the mistake-driven update $w := w + \eta * y * x$. Students should leave able to implement this update on a linearly separable toy set and to state the perceptron convergence theorem in one sentence: if a separating hyperplane exists, the algorithm finds one in a finite number of updates.*

Learning outcomes

- Write the perceptron forward pass and the mistake-driven weight update.
- Construct weight vectors for AND, OR and NAND on two binary inputs.
- State when the algorithm is guaranteed to halt.

Unit II: The XOR problem and multilayer networks (8 hours)

XOR is the canonical example of a function that is not linearly separable: the points (0,0) and (1,1) belong to class 0 while (0,1) and (1,0) belong to class 1, and no straight line separates them. Minsky and Papert used this to show the limits of a single layer. The fix is a hidden layer. A two-layer network can compute XOR by forming the intermediate features $x_1 \text{ AND } \text{NOT } x_2$ and $\text{NOT } x_1 \text{ AND } x_2$, then OR-ing those features. We introduce the multilayer perceptron as a composition of affine maps and pointwise activations, and we count parameters: for layers of size n_0, n_1, n_2 the weight matrices are n_1 by n_0 and n_2 by n_1 , plus a bias per hidden and output unit.

Learning outcomes

- Draw the XOR points and explain why one hyperplane is not enough.
- Hand-design a two-layer network that realises XOR with step activations.
- Count the parameters of a 4-5-4-2 fully connected network.

Unit III: Backpropagation and gradient descent (10 hours)

*Training replaces the step function with a differentiable activation so that a loss $L(w)$ can be differentiated. Mean squared error and binary cross-entropy are the two losses we use in this course. Backpropagation applies the chain rule from the output backwards: the gradient with respect to a layer's weights is the outer product of that layer's input and the incoming error signal. Stochastic gradient descent updates $w := w - \eta * \text{grad}$ on a minibatch. We discuss learning-rate schedules, vanishing gradients in saturating sigmoids, and why ReLU became the default hidden*

activation. Students implement backprop for a two-layer network on XOR and watch the loss fall.

Learning outcomes

- Derive dL/dw for a single linear unit with squared loss.
- Explain vanishing gradients for sigmoid activations in deep stacks.
- Train a two-layer MLP on XOR and report loss after 200 epochs.

Unit IV: Generalisation, overfitting and regularisation (6 hours)

A network that memorises the training set and fails on new points is said to overfit. We split data into train, validation and test, plot learning curves, and stop when validation loss rises. Regularisation methods in this unit are L2 weight decay, dropout, and data augmentation. We also introduce the bias-variance trade-off: too small a network underfits, too large a network overfits unless regularised. This unit closes the lecture course; labs continue with a small image classifier using the same ideas.

Learning outcomes

- Diagnose overfit vs underfit from a pair of learning curves.
- State the effect of increasing L2 weight decay on the effective capacity.
- Describe dropout at train time versus test time.

Laboratory list

- Lab 1: perceptron on a linearly separable 2-D set; plot the moving boundary.
- Lab 2: show XOR failure of a single layer; implement a 2-2-1 MLP.
- Lab 3: backprop from scratch in NumPy; compare eta in {0.01, 0.1, 1.0}.
- Lab 4: regularisation - dropout vs L2 on a noisy polynomial fit.